

L&T Files Assistant

An on-premise, AI-powered document intelligence platform for construction & engineering project documents — runs 100% locally, at zero API cost.

VERSION	1.0.0
PREPARED BY	NGXP Technologies
AUDIENCE	Executive & Technical Stakeholders
DATE	July 2026
CLASSIFICATION	Confidential

This document describes the architecture, capabilities and deployment of the L&T Files Assistant platform. Intended for the named recipient only.

▪ Contents

- [1 Executive Summary](#)
- [2 Platform at a Glance](#)
- [3 Key Capabilities](#)
- [4 System Architecture](#)
- [5 How It Works — The Intelligence Pipeline](#)
- [6 Document Processing & Supported Formats](#)
- [7 Technology Stack](#)
- [8 Security, Privacy & Data Governance](#)
- [9 Deployment & Infrastructure](#)
- [10 User Experience & Workflow](#)
- [11 API Reference](#)
- [12 Performance & Cost Model](#)
- [13 Roadmap & Extensibility](#)
- [14 Glossary](#)

1 Executive Summary

The **L&T Files Assistant** is a secure, AI-powered assistant that lets project teams ask plain-language questions about their engineering and construction documents — drawings, specifications, contracts, reports, spreadsheets and correspondence — and receive precise, source-cited answers in seconds.

Instead of manually searching through thousands of pages across hundreds of files, a user simply types a question — *"What is the total contract value across these tenders?"* or *"Show me the reinforcement schedule for the pier foundations"* — and the assistant reads the relevant documents, reasons across them, and returns an answer with citations back to the exact source files.

The platform's defining characteristic is that it runs **entirely on-premise**. Every AI model — the language model, the search embeddings, the reranker and the document OCR — executes locally on the organisation's own GPU hardware. No document content, query, or metadata ever leaves the network, and there are **no per-query API fees**. This makes it suitable for confidential tender, contract and design documentation that cannot be sent to third-party cloud services.

Why it matters

Project knowledge that today is locked inside PDFs and CAD exports becomes instantly queryable — while remaining fully within the organisation's security perimeter and at a fixed, predictable infrastructure cost.

100%

ON-PREMISE

\$0

PER-QUERY COST

25+

FILE FORMATS

40

API ENDPOINTS

What the platform delivers

- **Conversational document Q&A** with streaming answers and inline citations to source files.
- **Project-based organisation** — documents are grouped into projects and queried individually or across several at once.
- **Multi-step analytical reasoning** — comparisons, rankings, counting and computation across many documents, not just single-passage lookup.
- **Broad format coverage** including CAD drawings (DWG/DXF), Office files, scanned PDFs, images, audio/video and emails.
- **Exportable reports** — any conversation can be exported to a formatted Word or PDF document.
- **Full data sovereignty** — no external calls required at any point in normal operation.

2 Platform at a Glance

The L&T Files Assistant is a full-stack web application. Users work through a browser interface; behind it sits a Python service layer that orchestrates a complete Retrieval-Augmented Generation (RAG) pipeline, backed by a search engine, a task queue and a metadata database.

Dimension	Summary
Product type	On-premise AI document intelligence & conversational Q&A platform
Primary users	Project engineers, document controllers, contract & planning teams
Interface	Responsive web application (React) — Login, Projects, Documents, Chat
Core engine	Built on the open-source RAGFlow pipeline (hybrid search, GraphRAG, reranking) used unchanged
Language model	Local LLM via Ollama (e.g. qwen3 family) on GPU; optional OpenAI / Groq / Gemini
Deployment	Single-command Docker Compose stack, GPU-accelerated
Hosting model	Fully self-hosted / air-gap capable
Verified on	NVIDIA RTX 6000 Ada (48 GB VRAM)

Built on a proven foundation

Rather than reinventing the retrieval engine, the platform builds on **RAGFlow**, a mature open-source RAG framework, and uses its document-parsing and hybrid-search pipeline unchanged. The custom-built layer on top — a purpose-designed API, an intent router, an analytical reasoning agent, project management, reporting and the user interface — is what tailors the system specifically to L&T's engineering document workflows.

3 Key Capabilities

Ask in Plain Language

Natural-language questions over one project or many at once. Follow-up questions ("shorter", "and the second one?") are understood in context.

Grounded, Cited Answers

Every answer is generated only from retrieved document passages and names its source files — no fabricated facts.

Cross-Document Analysis

Compares, ranks, counts and computes over many records across multiple files using an autonomous reasoning agent.

Deep Document Parsing

OCR, page-layout analysis and table-structure recognition extract text and tables even from scanned or complex PDFs.

CAD & Office Ingestion

DWG/DXF drawings and PPT/DOC files are auto-converted to PDF so they flow through the same rich parsing pipeline.

Streaming Chat

Answers stream token-by-token over Server-Sent Events for an immediate, responsive experience.

Report Export

Export any conversation to a professionally formatted Word (.docx) or PDF document, tables and references included.

Project Workspaces

Documents are organised into projects (knowledge bases) with per-project stats and independent search scopes.

Admin & Access Control

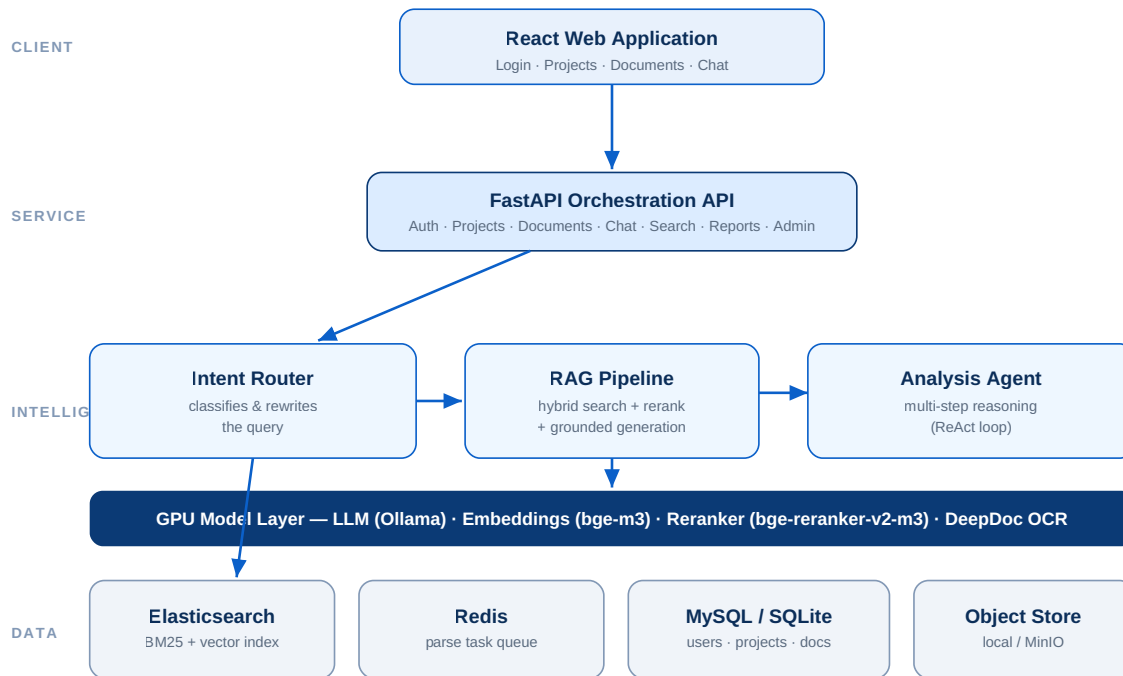
JWT authentication, admin/user roles, user management, model configuration and system health monitoring.

Intelligent handling of damaged tables

Engineering documents are dense with tables — bar-bending schedules, BOQs, load tables — and OCR frequently damages them. The platform includes a purpose-built, deterministic table processor that reconstructs table grids from the source markup (resolving merged/spanning cells) before the AI ever reads them, and applies strict discipline rules so that values from corrupted tables are flagged as unreliable rather than silently misreported. This directly targets one of the most common failure modes of generic document-AI tools on technical content.

4 System Architecture

The platform follows a clean separation between the browser front-end, an orchestration API, the RAG intelligence pipeline, and the supporting data services. All components run within the organisation's own environment.



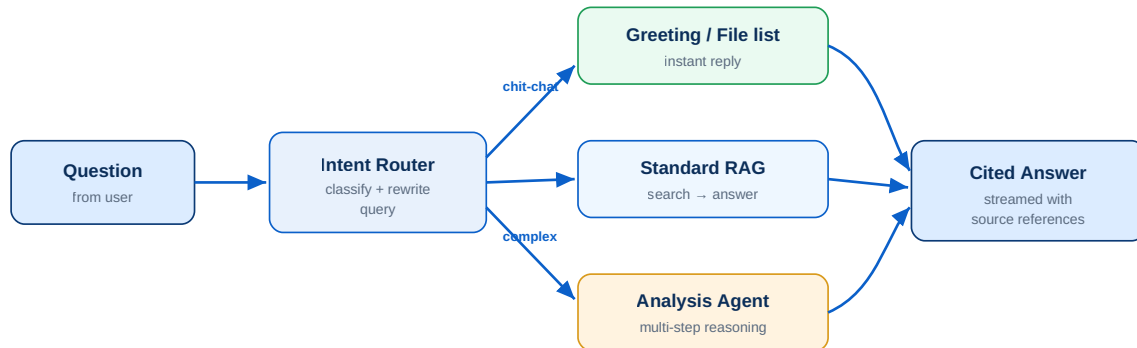
High-level architecture. All layers execute within the organisation's network; the GPU model layer requires no external services.

Layer responsibilities

Layer	Responsibility
Web Application	Browser UI for authentication, project & document management, and the chat workspace. Built with React, Vite, TypeScript and Tailwind CSS.
Orchestration API	FastAPI service exposing ~40 REST endpoints. Handles auth, request validation, and coordinates the intelligence pipeline.
Intent Router	A first-pass LLM step that understands what the user really wants, resolves follow-ups, and decides whether a request is small-talk, a file listing, a standard lookup, or a multi-step analytical task.
RAG Pipeline	The RAGFlow retrieval engine: hybrid keyword + semantic search, reranking, and answer generation strictly grounded in retrieved passages.
Analysis Agent	An autonomous reasoning agent for complex questions requiring several retrieval passes and exact computation.
Data services	Elasticsearch (search index), Redis (parsing queue), a relational database (metadata) and an object store (original files).

5 How It Works — The Intelligence Pipeline

When a user asks a question, the platform does not blindly search and answer. It runs a deliberate, multi-stage pipeline designed to maximise accuracy and minimise wasted effort.



Request routing. The router directs each question down the cheapest path that can answer it correctly, with automatic fallback to standard RAG if the agent cannot produce a trustworthy result.

Stage 1 — Intent routing

Before any search runs, an LLM router reads the conversation so far and classifies the latest message. It distinguishes genuine document questions from greetings and identity questions ("who are you?"), recognises requests to list files, and — crucially — resolves terse follow-ups such as "in 3 lines" or "and the second one?" against the previous turn. It rewrites the request into a clean, self-contained search query and decides whether the question needs the full analytical agent.

Stage 2 — Retrieval (hybrid search + rerank)

For standard questions, the RAGFlow pipeline performs **hybrid retrieval** — combining traditional keyword (BM25) matching with dense semantic vector search — then **reranks** the candidates with a cross-encoder model so the most relevant passages rise to the top. This two-signal approach retrieves both exact-term matches (part numbers, clause references) and conceptually related content.

Stage 3 — Grounded generation

The language model composes an answer using *only* the retrieved passages, and is explicitly instructed never to answer from its own training memory. Answers stream to the user in real time and carry citations back to the source documents, so every claim can be verified against the original file.

Stage 4 — The Analysis Agent (for complex questions)

Some questions cannot be answered by a single retrieval — "rank these five contractors by total quoted value" or "how many drawings reference this specification?" require gathering evidence across many passages and doing exact arithmetic. For these, the router engages an autonomous **Analysis Agent** that runs a bounded *think* → *act* → *observe* → *decide* loop:

- It can call two tools: **document search** (the same hybrid retrieval as above) and **compute** (exact, code-safe arithmetic — sum, average, min/max, count, sort).
- It searches repeatedly with refined queries until it has gathered all the evidence it needs, then computes precise results rather than estimating.

- It is bound by strict production guardrails: a hard cap on reasoning steps, a wall-clock time budget, and truncation of each observation to keep cost bounded.
- If it cannot produce a trustworthy answer within budget, it **falls back automatically** to the standard RAG path — the user always receives an answer, never an error.

Accuracy by design, not by chance

The agent is prohibited from inventing values, required to name its sources, and instructed to state honestly when the source documents simply do not contain the requested information. Arithmetic is performed by real code — never guessed by the model — so totals and rankings are exact.

6 Document Processing & Supported Formats

Uploaded files are processed by **DeepDoc**, RAGFlow's document-understanding engine, which performs OCR, page-layout analysis and table-structure recognition on the GPU. Parsing runs asynchronously through a Redis-backed task queue, with multiple parallel workers so large batches process quickly. The extracted content is chunked, embedded and indexed for retrieval.

Automatic format conversion

Formats that the parser cannot read directly are converted transparently on upload using LibreOffice in headless mode, so they benefit from the same full OCR, layout and table pipeline:

- **CAD drawings** — DWG and DXF files are auto-converted to PDF.
- **Office files** — PPT/PPTX and DOC/DOCX are converted to PDF for consistent layout-aware parsing.

Supported file types

Category	Formats
Documents	PDF, DOCX, DOC, XLSX, XLS, PPTX, PPT, TXT, MD, CSV, HTML, EPUB, JSON
Images	PNG, JPG, JPEG, BMP, TIFF, GIF
Media & audio	MP3, WAV, M4A, AAC, FLAC, MP4, AVI, MKV
CAD	DWG, DXF (auto-converted to PDF)
Email	EML, MSG

Maximum upload size and the exact accepted-extension list are configurable. Individual documents can be re-parsed on demand, and an entire project can be re-indexed by an administrator.

7 Technology Stack

Layer	Technology	Role
Front-end	React 18, Vite, TypeScript, Tailwind CSS	Web user interface
Front-end state / data	Zustand, TanStack Query, Axios, React Router	State management, data fetching, routing
Back-end API	Python, FastAPI, Uvicorn	REST API & orchestration
RAG engine	RAGFlow (DeepDoc, GraphRAG, retriever)	Parsing, indexing, retrieval
Language model	Ollama (local, GPU) — e.g. Qwen3; optional OpenAI / Groq / Gemini	Routing, reasoning & answer generation
Embeddings	BAAI <code>bge-m3</code> (in-process, GPU)	Semantic vector search
Reranker	BAAI <code>bge-reranker-v2-m3</code> (GPU)	Relevance re-ordering
Document OCR	DeepDoc on ONNX Runtime (CUDA)	OCR, layout & table recognition
Search index	Elasticsearch 8	Hybrid BM25 + vector index
Task queue	Redis 7	Asynchronous document parsing
Metadata DB	MySQL 8 (prod) / SQLite (dev) / PostgreSQL	Users, projects, documents, sessions
Object storage	Local filesystem or MinIO	Original uploaded files
Packaging	Docker & Docker Compose	Deployment & orchestration

Provider-flexible

A single configuration value (`LLM_PROVIDER`) is the source of truth for the chat model across the router, the agent and the RAG path. The default is fully local (Ollama on GPU), but the platform can be pointed at OpenAI, Groq or Gemini where a hosted model is preferred — without code changes.

8 Security, Privacy & Data Governance

Security was a primary design driver. The platform is built so that sensitive project documentation can be made searchable without ever leaving the organisation's control.

Control	How it is addressed
Data residency	All processing — parsing, embeddings, reranking, LLM inference — runs on local GPU hardware. In the default configuration no document content or query is transmitted to any external service.
Air-gap capability	Because every model runs locally, the platform can operate on a network with no outbound internet access.
Authentication	Token-based authentication using signed JWTs with a configurable expiry. Passwords are stored hashed.
Authorisation	Role separation between standard users and administrators; administrative endpoints (user management, system info, re-indexing) are restricted to admins.
Tenant isolation	Chat sessions and documents are validated against the owning tenant on every request, preventing cross-tenant access.
Transport & access	Served behind an Nginx reverse proxy; TLS/SSL and network access are controlled at the deployment layer.
Configuration hygiene	Secrets (JWT key, database and admin credentials, any API keys) are supplied via environment variables, not committed to code.

Deployment hardening checklist

Before go-live: replace the default admin password and JWT secret with strong random values, enable TLS at the proxy, restrict network exposure to trusted clients, and set database and search-engine credentials. These are standard production steps and are documented in the configuration guide.

9 Deployment & Infrastructure

The entire platform deploys with a single Docker Compose command. The production stack brings up every dependency — search engine, cache/queue, database, object store, the API, parallel parsing workers and the front-end — with the GPU wired into the components that need it.

Production stack (docker-compose)

Service	Purpose
elasticsearch	Hybrid keyword + vector search index
redis	Document-parsing task queue
mysql	Metadata database (users, projects, documents, sessions)
minio	Object storage for original files
backend	FastAPI API server — GPU-attached
task_executor (×3 replicas)	Parallel document parsing workers — GPU-attached, ~3× throughput
frontend	React web application served via Nginx

GPU acceleration

Every model runs on an NVIDIA GPU. The language model runs via Ollama as a native host process (for direct, reliable GPU access), which the containers reach over the internal host network; embeddings, reranking and DeepDoc OCR run on the GPU inside the containers. Running document OCR on the GPU delivers roughly an order-of-magnitude faster parsing than CPU-only, and a keep-alive setting keeps the language model resident so chats don't incur cold-start latency.

Component	Runs on
LLM (chat / router / agent)	GPU via Ollama (native host process)
Embeddings (bge-m3)	GPU, in-process (fp16)
Reranker (bge-reranker-v2-m3)	GPU, in-process (fp16)
DeepDoc OCR / layout / tables	GPU via ONNX Runtime (CUDA 12)

Prerequisites

A host with an NVIDIA GPU (validated on an RTX 6000 Ada, 48 GB VRAM), Docker with GPU support enabled, and Ollama running on the host with the chosen model pulled. Data persists across restarts in named Docker volumes.

10 User Experience & Workflow

The interface is deliberately simple, following a natural four-step flow.

Step	Screen	What the user does
1	Login	Signs in with their credentials to receive a secure session.
2	Projects	Creates or opens a project — a workspace grouping related documents — and sees document counts and status at a glance.
3	Documents	Uploads files (drag-and-drop), watches live parsing progress, and re-parses or downloads documents as needed.
4	Chat	Asks questions across one or more projects, reads streaming answers with citations, and exports the conversation to Word or PDF.

In the chat workspace

- Answers appear in real time and render rich Markdown — including tables — so structured results are easy to read.
- Citations link each answer back to its source documents.
- Sessions can span multiple projects, so a single conversation can reason across several document sets.
- The assistant handles conversational follow-ups naturally ("summarise that in three lines", "and the next one?").

11 API Reference

The back-end exposes roughly 40 REST endpoints, grouped by function. Interactive, self-documenting API docs are available at [/docs](#) when the service is running.

Group	Endpoints
Authentication	Login, logout, current user, change password
Users	Full user management (create, read, update, delete) — admin
Projects	Create / list / update / delete projects; per-project statistics
Documents	Upload (with DWG/Office auto-convert), list, status, re-parse, download, delete
Chat	Session management (CRUD) and streaming messages over Server-Sent Events
Search	Direct hybrid search; list indexed chunks for a project
Reports	Generate a Word/PDF report from a conversation; retrieve by session
Models	List providers; active chat / embedding / rerank models; connectivity test
System	Health, status, info, available parsers; project re-index — admin

Each endpoint validates authentication and, where relevant, tenant ownership before acting.

12 Performance & Cost Model

Cost

Because all AI runs locally, there is **no marginal per-query cost** and no usage-based cloud billing. The total cost of ownership is essentially the one-time GPU hardware plus ordinary hosting and operations — fully predictable regardless of how many questions users ask. This is a decisive advantage over hosted AI services, whose costs scale with usage and which cannot process confidential documents on-premise.

Performance characteristics

- **Parsing throughput** — GPU-accelerated OCR is roughly 10× faster than CPU, and three parallel parsing workers process document batches concurrently.
- **Responsiveness** — answers stream token-by-token; the model is kept warm to avoid cold-start delays.
- **Bounded agent cost** — the analysis agent is capped on both steps and wall-clock time, so worst-case latency and compute are predictable.
- **Efficient routing** — trivial requests (greetings, file lists) are answered instantly without invoking retrieval, and only genuinely complex questions engage the heavier agent path.

Predictable economics

Fixed infrastructure cost · zero per-query fees · scales with hardware, not with a vendor's price list.

13 Roadmap & Extensibility

The platform is built on an extensible foundation. The underlying RAGFlow engine already supports capabilities that can be surfaced in future releases, and the modular API makes new features straightforward to add.

Opportunity	Description
GraphRAG knowledge graphs	Build entity/relationship graphs across documents for deeper connected-knowledge queries (engine already supports this).
Additional messaging channels	The codebase includes connectors for chat channels (e.g. Teams-style bots), enabling Q&A from within collaboration tools.
Advanced analytics dashboards	Usage, coverage and document-health dashboards for administrators.
Fine-grained permissions	Project- and document-level access control for larger user populations.
Model upgrades	Drop-in adoption of newer local models as they are released, with no architectural change.
Scheduled ingestion	Automated pickup of documents from network shares or DMS systems.

Designed to grow

Because the intelligence layer is additive and every complex path falls back safely to standard retrieval, new capabilities can be introduced incrementally without risking the reliability of the core Q&A experience.

14 Glossary

Term	Meaning
RAG	Retrieval-Augmented Generation — the AI answers by first retrieving relevant document passages, then generating a response grounded in them.
Hybrid search	Combining keyword (BM25) and semantic vector search to retrieve both exact and conceptually related matches.
Reranker	A model that re-orders retrieved passages by true relevance to the question, improving answer precision.
Embedding	A numerical representation of text meaning used for semantic search.
LLM	Large Language Model — the AI that understands questions and writes answers.
OCR	Optical Character Recognition — extracting text from scanned pages and images.
Agent (ReAct)	An autonomous LLM process that reasons and calls tools in a loop to solve multi-step problems.
Knowledge base / Project	A grouped set of documents that can be searched together.
SSE	Server-Sent Events — the mechanism used to stream answers to the browser in real time.
DeepDoc	The document-understanding engine that performs OCR, layout analysis and table recognition.

L&T Files Assistant — Platform Documentation v1.0.0

Prepared by NGXP Technologies · July 2026 · Confidential. This document is provided for evaluation and reference purposes.